



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Lab Experiment – 12

Tic-Tac-Toe Game

Aim

To implement a Tic-Tac-Toe game using Python.

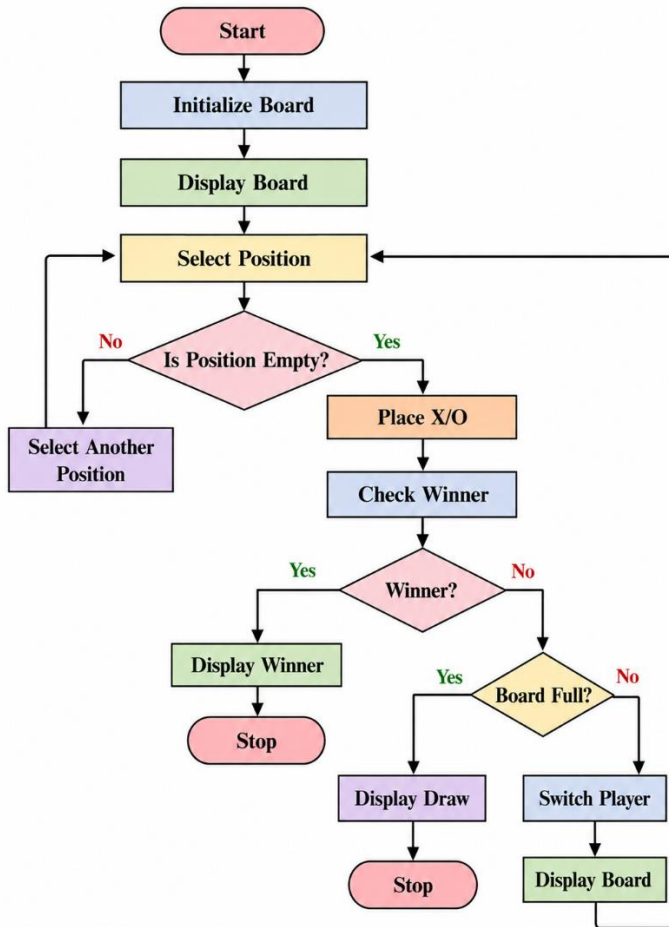
Objective

To develop a two-player Tic-Tac-Toe game where players take turns placing **X** and **O** on a 3×3 board.

Algorithm

1. Start the game.
2. Initialize a 3×3 board with empty spaces.
3. Display the board.
4. Ask the current player to select a position.
5. Place the player's symbol (x or o) in the selected position.
6. Check whether the player has won.
7. If a player wins, display the winner and stop.
8. Check whether the board is full.
9. If the board is full, declare the game as a draw.
10. Otherwise, switch to the other player.
11. Repeat the process until the game ends.

FlowChart



Code

```
def print_board(board):
```

```
    print("\n")
```

```
    print(" | ".join(board[0:3]))
```

```
    print("--+---+--")
```

```
    print(" | ".join(board[3:6]))
```

```
    print("--+---+--")
```

```
    print(" | ".join(board[6:9]))
```

```
    print()
```

```
def check_winner(board, player):
```

```
winning_positions = [
```

```
    (0, 1, 2),
```

```
    (3, 4, 5),
```

```
    (6, 7, 8),
```

```
    (0, 3, 6),
```

```
    (1, 4, 7),
```

```
    (2, 5, 8),
```

```
    (0, 4, 8),
```

```
    (2, 4, 6)
```

```
]
```

```
for a, b, c in winning_positions:
```

```
    if board[a] == board[b] == board[c] == player:
```

```
        return True
```

```
return False
```

```
board = [" "] * 9
```

```
player = "X"
```

```
for turn in range(9):
```

```
    print_board(board)
```

```
    position = int(input(
```

```
        f"Player {player}, enter position (1-9): "
```

```
    )) - 1
```

```
    if position < 0 or position > 8 or board[position] != " ":
```

```
print("Invalid position. Try again.")  
continue
```

```
board[position] = player
```

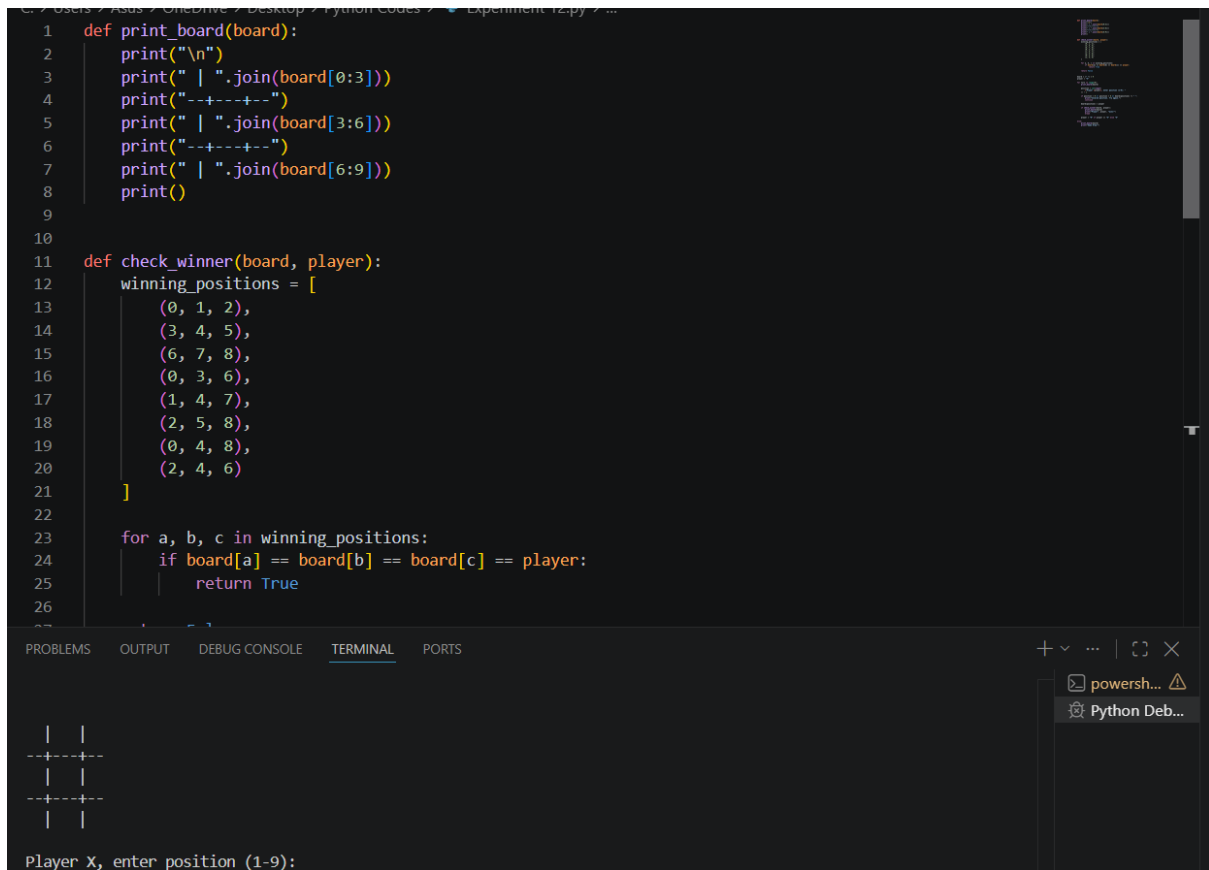
```
if check_winner(board, player):  
    print_board(board)  
    print("Player", player, "wins!")  
    break
```

```
player = "O" if player == "X" else "X"
```

```
else:
```

```
    print_board(board)  
    print("Game Draw!")
```

Output



```
1 def print_board(board):
2     print("\n")
3     print(" | ".join(board[0:3]))
4     print("--+---+--")
5     print(" | ".join(board[3:6]))
6     print("--+---+--")
7     print(" | ".join(board[6:9]))
8     print()
9
10
11 def check_winner(board, player):
12     winning_positions = [
13         (0, 1, 2),
14         (3, 4, 5),
15         (6, 7, 8),
16         (0, 3, 6),
17         (1, 4, 7),
18         (2, 5, 8),
19         (0, 4, 8),
20         (2, 4, 6)
21     ]
22
23     for a, b, c in winning_positions:
24         if board[a] == board[b] == board[c] == player:
25             return True
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Player X, enter position (1-9):

```
| |
--+---+--
| |
--+---+--
| |
```

Result

The Tic-Tac-Toe game was successfully implemented in Python.

Conclusion

The program successfully implements a two-player Tic-Tac-Toe game by managing player turns, validating positions, checking winning combinations, and identifying a draw when the board becomes full.